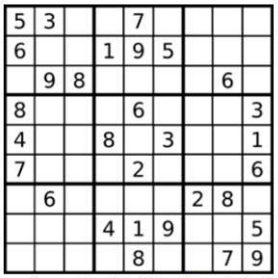
a.) I am using Chatgpt and Gemini

b.) I sourced them using my laptop

c.)

1. Use two GenAI's to generate C++ code using objects, not functions, to implement the following requirements (Note: You may use either ChatGPT or CoPilot as one of your choices, but not both. They are essentially the same GenAI.):
   - Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks.
   - A typical puzzle is shown here (image credit: Wikimedia Commons):
     - 

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

   - Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints:
     - Each digit from 1 to 9 must appear exactly once in each row.
     - Each digit from 1 to 9 must appear exactly once in each column.
     - Each digit from 1 to 9 must appear exactly once in each block.
   - Published puzzles are usually constructed to have a unique solution. This requires that at least 17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve.
   - The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that.
   - It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it's possible to gradually fill in the entire grid.
   - That is not the approach we're going to use. Instead, we're going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by "slower to run," we mean it'll finish in a few tenths of a second rather than a few milliseconds.)
   - Here's the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists

     (all candidates report no solution, which again implies we took a wrong path somewhere earlier).
   - This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.
   - You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
   - Write a **recursive object-oriented** program to find the solution to the five puzzle files.
   - The output for each solution, should include:
     - The puzzle file name (e.g., puzzle1.txt)
     - The puzzle from the file printed out.
     - The solution to the puzzle printed out or if no solution is found, "No solution found" printed out.
   - The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.
   - To help you debug your program, solution1.txt is a solution to puzzle1.txt.

e.
Correctness

Chatgpt – Checks rows, columns and the 3x3 blocks, uses recursive backtracking to find all the possible solutions, prints original puzzle, all solutions, and when no solution is found prints 'No solution found', also checks that the input contains 81 cells.
Gemini – Correctly checks the rows, columns, and 3x3 blocks, uses recursive backtracking, reads all the puzzle files.

Execution Time
Chatgpt – Uses a function to find the next cell, checks the validity of a number before placing it, uses recursion to find each solution
Gemini – Skips cells that already have numbers, checks what the possible numbers are before it places them, doesn't keep searching for the next empty cell

Space Complexity
Both have the same space complexity, but Gemini has a possible advantage because it doesn't constantly search for the next empty cell.

Maintainability
Chatgpt – Methods made have descriptive titles, separated loading, printing and solving into different methods, uses comments
Gemini – Uses descriptive titles for methods, uses comments in recursive algorithm.

f. I would pick Chatgpt since it has less errors, clearer comments, and better file validation. I would however make sure it is checking for numbers that conflict, make better file validation and also make it so that the execution time is a bit better.

g.
Correctness – Added a method that checks for conflicting numbers in the puzzle, also updated the file validation to ensure only 81 cells are provided.

Execution Time – Stopped the repeated searching for empty cells and instead have the program just move through the cells using grid positions.

Space Complexity – Stayed the same

Maintainability – Added comments for the new method.

d.) First is **chatgpt**

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

class Sudoku
{
private:
  int grid[9][9];
  int solutionCount;

  // Check whether a number can be placed in a cell.
  bool isValid(int row, int col, int num)
  {
    // Check the row.
    for (int c = 0; c < 9; c++)
    {
      if (grid[row][c] == num)
        return false;
    }

    // Check the column.
```

```
    for (int r = 0; r < 9; r++)
    {
        if (grid[r][col] == num)
            return false;
    }

    // Find the top-left corner of the 3x3 block.
    int startRow = (row / 3) * 3;
    int startCol = (col / 3) * 3;

    // Check the 3x3 block.
    for (int r = startRow; r < startRow + 3; r++)
    {
        for (int c = startCol; c < startCol + 3; c++)
        {
            if (grid[r][c] == num)
                return false;
        }
    }

    return true;
}

// Find the next empty cell.
bool findEmptyCell(int &row, int &col)
{
```

```
    for (row = 0; row < 9; row++)
    {
        for (col = 0; col < 9; col++)
        {
            if (grid[row][col] == 0)
                return true;
        }
    }

    return false;
}

// Recursive backtracking search.
void searchSolutions()
{
    int row, col;

    // No empty cells means a complete solution was found.
    if (!findEmptyCell(row, col))
    {
        solutionCount++;

        cout << "\nSolution " << solutionCount << ":\n";
        printPuzzle();

        // Return to the previous recursive call to
```

```
        // continue searching for other solutions.
        return;
      }

      // Try every possible digit.
      for (int num = 1; num <= 9; num++)
      {
        if (isValid(row, col, num))
        {
          grid[row][col] = num;

          // Search for all solutions.
          searchSolutions();

          // Undo this choice and try another digit.
          grid[row][col] = 0;
        }
      }
    }

public:
  // Initialize an empty grid.
  Sudoku()
  {
    solutionCount = 0;
```

```
    for (int r = 0; r < 9; r++)
    {
        for (int c = 0; c < 9; c++)
        {
            grid[r][c] = 0;
        }
    }
}

// Read a puzzle file.
bool loadPuzzle(string filename)
{
    ifstream inputFile(filename);

    if (!inputFile.is_open())
        return false;

    char ch;
    int count = 0;

    // Read digits and underscores, ignoring whitespace.
    while (inputFile.get(ch) && count < 81)
    {
        if (ch >= '1' && ch <= '9')
        {
            grid[count / 9][count % 9] = ch - '0';
```

```
            count++;
        }
        else if (ch == '_')
        {
            grid[count / 9][count % 9] = 0;
            count++;
        }
        // Ignore spaces, newlines, and other separators.
    }

    inputFile.close();

    // The file must contain exactly 81 cell symbols.
    return count == 81;
}

// Print the current grid.
void printPuzzle()
{
    for (int r = 0; r < 9; r++)
    {
        if (r % 3 == 0)
            cout << "+-------+-------+-------+\n";

        for (int c = 0; c < 9; c++)
        {
```

```
            if (c % 3 == 0)
                cout << "| ";

            if (grid[r][c] == 0)
                cout << "_ ";
            else
                cout << grid[r][c] << " ";
        }

        cout << "|\n";
    }

    cout << "+-------+-------+-------+\n";
}

// Solve the puzzle and print every solution.
void solve()
{
    solutionCount = 0;

    searchSolutions();

    if (solutionCount == 0)
    {
        cout << "\nNo solution found\n";
    }
```

```
        else
        {
            cout << "\nTotal solutions found: "
                 << solutionCount << "\n";
        }
    }
};

int main()
{
    for (int i = 1; i <= 5; i++)
    {
        string filename = "puzzle" + to_string(i) + ".txt";

        cout << "\n==================================\n";
        cout << "Puzzle file: " << filename << "\n";
        cout << "==================================\n";

        Sudoku puzzle;

        if (puzzle.loadPuzzle(filename))
        {
            cout << "\nOriginal puzzle:\n";
            puzzle.printPuzzle();

            cout << "\nSearching for solutions...\n";
```

```
        puzzle.solve();
      }
      else
      {
        cout << "Could not read " << filename
          << ". Check that the file
exists and "
          << "contains 81 cell symbols.\n";
      }
    }

    return 0;
}
```

gemini code

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class SudokuBoard {
private:
    char grid[9][9];

    // Helper method to check if placing a digit is valid according to Sudoku rules
```

```
bool isValid(int row, int col, char num) const {
   for (int i = 0; i < 9; ++i) {
      // Check row and column
      if (grid[row][i] == num) return false;
      if (grid[i][col] == num) return false;

      // Check 3x3 sub-grid / block
      int blockRow = 3 * (row / 3) + i / 3;
      int blockCol = 3 * (col / 3) + i % 3;
      if (grid[blockRow][blockCol] == num) return false;
   }
   return true;
}

// Recursive backtracking method to find all solutions
bool solveRecursive(int row, int col, int& solutionCount) {
   // If we have moved past the last row, we found a solution
   if (row == 9) {
      solutionCount++;
      std::cout << "\nSolution #" << solutionCount << ":\n";
      printBoard();
      return true; // Return true to keep searching for other solutions
   }

   // Move to the next cell
   int nextRow = (col == 8) ? row + 1 : row;
```

```
        int nextCol = (col == 8) ? 0 : col + 1;

        // If the current cell is already filled, skip to the next
        if (grid[row][col] != '_') {
            return solveRecursive(nextRow, nextCol, solutionCount);
        }

        bool found = false;
        // Try digits '1' through '9'
        for (char digit = '1'; digit <= '9'; ++digit) {
            if (isValid(row, col, digit)) {
                grid[row][col] = digit;
                if (solveRecursive(nextRow, nextCol, solutionCount)) {
                    found = true;
                }
                grid[row][col] = '_'; // Backtrack
            }
        }
        return found;
    }

public:
    // Constructor initializes the grid
    SudokuBoard() {
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
```

```
            grid[i][j] = '_';
        }
    }
}

// Load puzzle from a file
bool loadFromFile(const std::string& filename) {
    std::ifstream file(filename);
    if (!file.is_open()) {
        return false;
    }
    for (int i = 0; i < 9; ++i) {
        for (int j = 0; j < 9; ++j) {
            file >> grid[i][j];
        }
    }
    return true;
}

// Print the Sudoku board
void printBoard() const {
    for (int i = 0; i < 9; ++i) {
        for (int j = 0; j < 9; ++j) {
            std::cout << grid[j] << " ";
        }
        std::cout << "\n";
```

```
        }
    }

    // Solve the puzzle and print results
    void solve(const std::string& filename) {
        std::cout << "================================\n";
        std::cout << "Puzzle File: " << filename << "\n";
        std::cout << "Original Puzzle:\n";
        printBoard();

        int solutionCount = 0;
        bool hasSolution = solveRecursive(0, 0, solutionCount);

        if (!hasSolution || solutionCount == 0) {
            std::cout << "\nNo solution found\n";
        }
        std::cout << "================================\n\n";
    }
};

int main() {
    // Process puzzle files: puzzle1.txt through puzzle5.txt
    for (int i = 1; i <= 5; ++i) {
        std::string filename = "puzzle" + std::to_string(i) + ".txt";
        SudokuBoard puzzle;
```

```
        if (puzzle.loadFromFile(filename)) {
            puzzle.solve(filename);
        } else {
            std::cout << "Could not open file: " << filename << "\n";
        }
    }

    return 0;
}
```